Assignment 2

Stochastic Processes Simulations, 58215

Sam Moeini
sam.moeini01@gmail.com
samo0171

André Johansson
andre.johansson12@gmail.com
anyp0002

Ellinor Olsson
Ellinor.c.olsson@gmail.com
elol0112

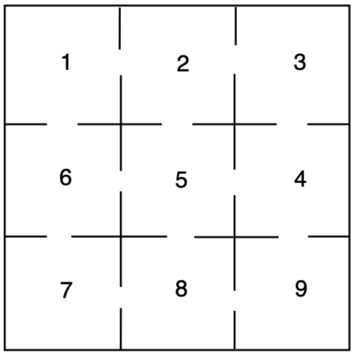
7 december 2023

Innehåll

1 Task 1: The White Rat

1.1 Task Formulation

A white rat is put into the random cell of the maze on the Figure below. There are nine compartments with connections between the compartments, as indicated. At each time step, the rat moves through the compartments at random. That is, if there are k ways to leave a compartment, it chooses each of these with equal probability.



a) Consider long period of time (n=100 000 time steps). Use simulations to estimate the proportion of time spent by the rat in each cell.

b) Use Matlab to compute the theoretical solution of the problem. Present necessary reasoning guaranteeing the existence of the solution presented.

c) Assume now that a rat is placed in the position 5 initially (at time 0). Based on 1000 trials, what is the estimate of the expected time for the rat to return to its initial position? What is the standard error associated with your point estimate?

1.2 Theory

Definition 1 (Irreducible Markov Chain) *Markov Chain* X_n, n = 0, 1, 2, . . . *with space state*$\{1, 2, ..., N\}$ *is called irreducible if each state* j, j$\in \{1, 2, ..., N\}$ *can be reached in a finite time from any other state.*

Theorem 1 Let X_n, n = 1, 2, . . . *with space state* 1, 2, . . . , N *be an* ***irreducible*** *Markov Chain. Then there exist a stationary distribution* π = (π_1, π_2, . . . , π_N) *such that*

$$\pi_j = \lim_{n\to\infty} \frac{1}{n} \sum_{k=1}^{n} \mathbf{1}(X_k = j),$$

Where π is interpreted as the proportion of time that the chain spends in state j in the long run and the limit represents a limit in probability. with additional condition

$$\sum_{i=1}^{N} \pi_i = 1$$

Which guarantees the uniqueness of the solution. The unique distribution π is also a solution of

$$\pi P = \pi$$

1.3 Task 1 a)

We are asked to use simulations to estimate the proportion of time spent in each cell by the rat. We do this by creating a 3x3 matrix with 9 cells labeled accordingly. Then we use the built in *"unidrnd()"* function to generate a random discrete numbers to randomize the rats movement. Depending on how many different doors the rat can leave that specific cell from. Let´s say the rat is in cell 5, it then has 4 options and we would then generate 4 random discrete variables $\in \{1, 2, 3, 4\}$. However if the rat is located in any other cell it can have either 2 or 3 ways of leaving its compartment and we would then generate two or three discrete variables to randomize its movement. We then run through a loop 100 000 times (*steps*) to estimate the proportion of time that the rat spends in each cell. The results are obtained in the end by counting the number of times it passed through each room.

$$P = \begin{bmatrix} 0 & \frac{1}{2} & 0 & 0 & 0 & \frac{1}{2} & 0 & 0 & 0 \\ \frac{1}{3} & 0 & \frac{1}{3} & 0 & \frac{1}{3} & 0 & 0 & 0 & 0 \\ 0 & \frac{1}{2} & 0 & \frac{1}{2} & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & \frac{1}{3} & 0 & \frac{1}{3} & 0 & 0 & 0 & \frac{1}{3} \\ 0 & \frac{1}{4} & 0 & \frac{1}{4} & 0 & \frac{1}{4} & 0 & \frac{1}{4} & 0 \\ \frac{1}{3} & 0 & 0 & 0 & \frac{1}{3} & 0 & \frac{1}{3} & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & \frac{1}{2} & 0 & \frac{1}{2} & 0 \\ 0 & 0 & 0 & 0 & \frac{1}{3} & 0 & \frac{1}{3} & 0 & \frac{1}{3} \\ 0 & 0 & 0 & \frac{1}{2} & 0 & 0 & 0 & \frac{1}{2} & 0 \end{bmatrix}$$

The simulations that we ran generated the following results over 100 000 iterations. Each value represents time spent in that cell as a percentage.

Cell 1: 8.4280
Cell 2: 12.5010
Cell 3: 8.3370
Cell 4: 12.4810
Cell 5: 16.6470
Cell 6: 12.5860

Cell 7: 8.2890
Cell 8: 12.4320
Cell 9: 8.2990

Algorithm Task 1 a)

1. Create the 3x3 Matrix
2. Randomize the starting position
3. Save the rat's position and randomize its next move
4. Repeat step 3 $n = 100,000$ times
5. Check how many times the rat visited each cell and create a matrix with the corresponding values for each cell divided by $n = 100,000$
6. Display the matrix

1.4 Task 1 b)

We are to use Matlab to compute the theoretical solution of the problem posed in task 1 a). The first thing we need to examine is whether or not the chain is irreducible. By examination of the maze we can come to the conclusion that the markov chain has to be irreducible since it is possible to go to every cell from every cell in a finite number of steps. This can easily be confirmed by showing:

$1 \to 2 \to 3 \to 4 \to 5 \to 6 \to 7 \to 8 \to 9 \to 8 \to 7 \to 6 \to 5 \to 4 \to 3 \to 2 \to 1$

Hence, we have proven that the Markov chain is irreducible. Therefore, it must have a unique stationary distribution. Furthermore, we need to solve the expression $\pi \cdot P = \pi$, where P is the transition matrix described under task 1a), and $\pi \in (\pi_1, \pi_2, \ldots, \pi_N)$ represents the time spent in each cell.

We solve this by creating a matrix A which is calculated by subtracting the identity matrix from the transpose of matrix P, $(A = P' - I)$. Matrix A then becomes

$$A = \begin{bmatrix} -1 & \frac{1}{3} & 0 & 0 & 0 & \frac{1}{3} & 0 & 0 & 0 \\ \frac{1}{2} & -1 & \frac{1}{2} & 0 & \frac{1}{4} & 0 & 0 & 0 & 0 \\ 0 & \frac{1}{3} & -1 & \frac{1}{3} & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & \frac{1}{2} & -1 & \frac{1}{4} & 0 & 0 & 0 & \frac{1}{2} \\ 0 & \frac{1}{3} & 0 & \frac{1}{3} & -1 & \frac{1}{3} & 0 & \frac{1}{3} & 0 \\ \frac{1}{2} & 0 & 0 & 0 & \frac{1}{4} & -1 & \frac{1}{2} & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & \frac{1}{3} & -1 & \frac{1}{3} & 0 \\ 0 & 0 & 0 & 0 & \frac{1}{4} & 0 & \frac{1}{2} & -1 & \frac{1}{2} \\ 0 & 0 & 0 & \frac{1}{3} & 0 & 0 & 0 & \frac{1}{3} & -1 \end{bmatrix}$$

Then, we set π equal to the null space of matrix A using MATLAB's "null()" function. The null space represents the set of vectors that, when multiplied by matrix A, result in the zero vector. In the context of Markov chains, finding a vector in the null space of $(P' - I)$ corresponds to finding the stationary distribution. We then get

$$\pi = \begin{bmatrix} 0.2425 & 0.3638 & 0.2425 & 0.3638 & 0.4851 & 0.3638 & 0.2425 & 0.3638 & 0.2425 \end{bmatrix}$$

After obtaining the vector π from the null space, we normalize it to ensure that it represents a valid probability distribution that sums up to 1. This is done by dividing π by the sum of its elements using MATLAB's built in "sum()" function. We then get the theoretical solution as shown below.

$$\pi = \begin{bmatrix} 0.0833 & 0.1250 & 0.0833 & 0.1250 & 0.1667 & 0.1250 & 0.0833 & 0.1250 & 0.0833 \end{bmatrix}$$

Algorithm Task 1 b)

1. Set P equal to the transition matrix
2. Set I to the identity matrix
3. Set A equal to the transpose of $P - I$
4. Set π equal to the null space of A
5. Set π equal to π divided by the sum of π

1.5 Task 1 c)

In this task the rat starts off in cell 5 and we want to calculate the expected time (*number of steps*) for the rat to return to its initial position over 1000 trials. To do this we have created a while loop that runs while the rat is not in position 5 with the exception of the starting position. Inside this while loop we have a counter that counts the ammount of steps needed before the rat returns. Once it returns we have completed 1 trial. We then repeat this process 1000 times using a for loop whilst simultaneously storing each return time in an array. This is needed to solve the latter part of the question where we are looking for the standard error associated with our point estimate. Then using the stored values we can easily calculate the estimated return time and the standard error with the help of some of matlabs built in functions.

Algorithm Task 1 c)

1. Create the 3x3 matrix
2. Set the starting position to cell 5
3. Randomize the rat's movements until it has returned to cell 5
4. Run 1000 trials of the simulation
5. Create an array that holds each unique return time
6. Calculate the expected return time
7. Calculate the standard error
8. Return the expected return time and the standard error

2 Task 2: Back to origin

2.1 Task Formulation

Consider a Brownian motion in two dimensions. This motion is named after the botanist Robert Brown, who first described the phenomenon, while looking through a microscope at pollen particle immersed in water. Mathematically, the process observed by Robert Brown can be described with help of two independent Brownian motions $\{W_1(t), t \geq 0\}$ and $\{W_2(t), t \geq 0\}$ with common parameter σ. The position of the pollen (particle) in Euclidean space as time progresses can be then described

$$(x(t), y(t)) = (W_1(t), W_2(t)), \qquad t \geq 0$$

so movement in each coordinate is simply described by one dimensional Brownian motion. Observe that we naturally have that the motion process starts in the origin (0,0). Now, assume that we put an absorbing bounding box determined by $x = -1$, $x = 1$, $y = -1$ and $y = 1$ (square with edges of length 2). When the process touches the boundaries it stops.

a) Assume that for both processes we have $\sigma = 1$. Describe the algorithm to simulate the above mentioned process. Plot an example of realisation of the process together with boundary. Use time discretisation step not bigger than $dt = 0.0001$ (you can decrease it further, depending on your computer capacity).

b) Using the result of a) construct a Monte Carlo algorithm to estimate the expected life time of such process (i.e., time before it hits the boundary). Provide a 95% confidence interval for the estimate. Choose dt and number of replicates N suitably to your computer capacity, but not bigger than $dt = 0.001$ and $N = 1000$, respectively.

2.2 Theory

Definition 2 (Brownian Motion) *Random process $\{W(t), t \geq 0\}$ is called Brownian motion with parameter σ^2 if the following is satisfied:*

1. $W(0) = 0$
2. $W(t)$ *has independent increments*
3. $W(t)$ *has stationary increments, such that, for* $0 \leq s \leq t$ *we have*

$$W(t) - W(s) \sim \mathcal{N}(0, \sigma^2(t - s))$$

If $\sigma = 1$, then we say that we have a standard Brownian motion.

Definition 3 (Brownian Motion) *Due to the continuous nature of Brownian Motion exact simulation is immposible. Therefore the most effective strategy is to employ small time steps and replicate the process multiple times using the Monte Carlo method. This approach approximates Brownian Motion using small time increments denoted as Δt which ensures that as Δt approaches zero the approximation increasingly resembles the actual process. The process simulated with a time step Δt can be expressed as:*

$$W(\Delta t) = W(k\Delta t) - W((k-1)\Delta t) \sim \mathcal{N}(0, \sigma^2 \Delta t)$$

Furthermore for simulating Brownian Motion over the interval $[0, T]$ the required number of independent increments is given by $N = \frac{T}{\Delta t}$.

The Monte Carlo method is utilized for simulating and approximating problems influenced by randomness, while retaining a deterministic component. In this scenario, it serves to estimate the sampled mean. By repeatedly executing a process and applying the Central Limit Theorem, the accumulated data is expected to converge to a normal distribution, making the average an unbiased estimator for the population mean.

Theorem 2 (Law of Large Numbers) *Let $X_1, X_2, \ldots, X_N$ be independent, identically distributed variables with $\mu = E(X_i)$ and $V(X_i) < \infty$. Define $\overline{X}_N = \frac{1}{N}\sum_{i=1}^{N} X_i$. Then, we know that*

$$\lim_{N\to\infty} \overline{X}_N = \mu.$$

Adjusting distribution: To simulate a random variable with a normal distribution $\mathcal{N}(0, \sigma^2\Delta t)$ we can simulate a standard normal variable $X \sim \mathcal{N}(0,1)$ and scale it to match the desired variance given by: $\sigma\sqrt{\Delta t}X \sim \mathcal{N}(0, \sigma^2\Delta t)$.

2.3 Task 2 a)

In order simulate the process described in the task description we used the theory given above. Our step-size and standard deviation was given. The two brownian motions represent our x and y-coordinates. Using matlabs pseudo-random numbers from standard normal distribution we could input them in the equation presented in 'Adjusting distribution' to get the correct distribution and move one step-size. As long as the step-size dont lead the particle outside our boundry of x=-1, x=1 and y=-1, y=1 we keep going. In order to plot this process we save each step-size into a cumulative sum that is then plotted when one of our particles touches the boundry.

Algorithm Task 2 a)

1. – Set σ to 1 (scale factor for Brownian motion).
   – Set the time step dt to 0.0001 (time increment for the simulation).
   – Initialize $x(1)$ and $y(1)$ to 0 (starting coordinates of the particle).
   – Set i to 2 (index for the next position).

2. – Run an infinite loop (`while true`) to simulate the Brownian motion.

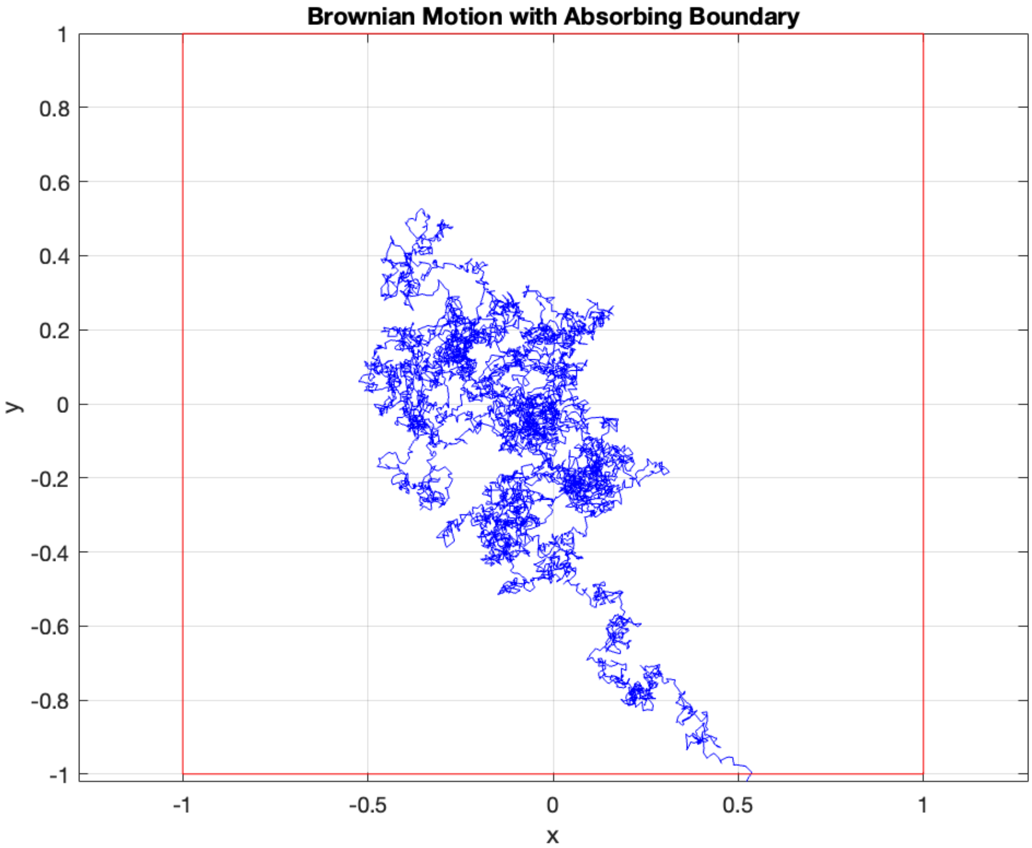
3. – Inside the loop, increment the x-coordinate: $x(i) = x(i-1) + \sigma\sqrt{dt} \cdot \texttt{randn}$.
   – Increment the y-coordinate similarly: $y(i) = y(i-1) + \sigma\sqrt{dt} \cdot \texttt{randn}$.

4. – If the absolute value of $x(i)$ or $y(i)$ is greater than or equal to 1, exit the loop (particle has reached the boundary).

5. – If the boundary condition is not met, increment i by 1 to calculate the next position.

6. – Plot the trajectory of the particle using the coordinates stored in x and y from the first position up to the ith position.

Results



Figur 1: Realisation of the brownian motion showing 7609 iterations occuring before the simulation terminated

2.4 Task 2 b)

The focus of this task is to determine the duration until the particle reaches the boundary. This duration is tracked with a step-counter that records only when the boundary is reached. To estimate the expected time for this event we use the Monte Carlo method with 1000 iterations to simulate the process and compute the average time from these simulations.

The 95% confidence interval is calculated from the average time denoted by $\bar{X}$ which serves as the point estimate. The standard error is calculated by using Matlab's built-in functions for variance and computing it to the following formula:

$$SE = \sqrt{\frac{\bar{\sigma}^2}{N}}$$

The confidence interval then becomes:

$$(\bar{X} - 1.96 \cdot SE < \mu < \bar{X} + 1.96 \cdot SE)$$

Algorithm Task 2 b)

1. – Set the time step dt to 0.001.
   – Set the number of simulations N to 1000.
   – Initialize an array x with length N with all elements set to zero.
   – Set σ to 1

2. – For each simulation iteration j from 1 to N:
   (a) Initialize $W1$ and $W2$ to 0 (starting positions).
   (b) Set i to 0 (step counter for each simulation).
   (c) While $|W1| < 1$ and $|W2| < 1$ (particle has not reached the boundary):
      * Increment $W1$ and $W2$ by a normally distributed random step scaled by $\sigma\sqrt{dt}$.
      * Increment i by 1.
   (d) Assign $x(j)$ to $i \times dt$ (store the time taken to hit the boundary in the j-th simulation).

3. – Calculate the mean of array x, denoted X_{bar}, representing the expected lifetime.
   – Compute the standard error as $SE = \frac{\text{std}(x)}{\sqrt{N}}$.
   – Calculate the 95% confidence interval upper and lower bounds:
      * $upper = X_{\text{bar}} + 1.96 \times SE$.
      * $lower = X_{\text{bar}} - 1.96 \times SE$.

Results

- **Expected Lifetime:** 0.608699
- **95% Confidence Interval:** [0.581782, 0.635616]

3 Task 3: Decomposition

3.1 Task Formulation

Consider a non-homogeneous Poisson process $\{N(t), t \geq 0\}$ *with intensity function*

$$\lambda(t) = 3 + \frac{4}{\sqrt{t+2}}, \qquad t > 0$$

We want to simulate the process on time interval $[0, 10]$.

a) *Decompose the process into a homogeneous and non-homogeneous Poisson process. Construct two algorithms to simulate the homogeneous and non-homogeneous component of the process, respectively.*

b) *Use the results from a) to simulate trajectories of* $\{N(t), t \in [0, 10]\}$. *Plot 50 realisations of the process on a common plot together with the corresponding mean value function.*

c) *Use 10000 simulated realisations of the process to estimate* $E(N(10))$ *and* $V(N(10))$. *Do your estimates coincide with theoretical values (reason what those are)?*

3.2 Theory

Definition 4 (Homogeneous Poisson Process): *Suppose that events are occurring at random time points and let* $N(t)$ *denote the number of events in the interval* $[0, t)$. *These events create a homogeneous Poisson process with rate* $\lambda > 0$ *if*

1. $N(0) = 0$
2. *The number of events occurring in disjoint intervals are independent (independent increments).*
3. *The distribution of the number of events that occurs in a given interval depends only on the length of the interval and not on its location (stationary increments).*
4. *On short intervals, the probability of events is proportional to the interval length.*
$$\lim_{h \to 0} \frac{P(N(h) = 1)}{h} = \lambda \quad \textit{[For } h \approx 0,\ P(N(h) = 1) \approx \lambda h\textit{]}$$
5. *No two events at the same time.*
$$\lim_{h \to 0} \frac{P(N(h) > 1)}{h} = 0 \quad \textit{[For } h \approx 0,\ P(N(h) > 1) \approx 0\textit{]}$$

Theorem 3 (Properties of Poisson Process): *Consider a Poisson process* $\{N(t), t \geq 0\}$ *with intensity* λ. *Then:*

1. *The number of occurrences in a time interval of length $t > 0$ is Poisson distributed with parameter λt, i.e.,*

$$N(s+t) - N(s) \sim Po(\lambda t), \quad \forall s > 0$$

2. *The interarrival times are independent and identically exponentially distributed with parameter λ.*

Theorem 4 (Superposition Property): *Let $\{N_1(t), t \geq 0\}$ and $\{N_2(t), t \geq 0\}$ be two independent Poisson processes with intensities λ_1 and λ_2, respectively. Then $N(t) = N_1(t) + N_2(t)$ is a Poisson process with intensity $\lambda = \lambda_1 + \lambda_2$.*

Definition 5 (Non-homogeneous Poisson Process): *Suppose that events are occurring at random time points and let $N(t)$ denote the number of events in the interval $[0, t)$. These events create a non-homogeneous Poisson process with intensity $\lambda(t) \geq 0$ if*

1. $N(0) = 0$
2. *The number of events occurring in disjoint intervals are independent (independent increments).*
3. $$\lim_{h \to 0} \frac{P(N(t+h) - N(t) = 1)}{h} = \lambda(t)$$
4. $$\lim_{h \to 0} \frac{P(N(t+h) - N(t) > 1)}{h} = 0$$

For a non-homogeneous Poisson process $\{N(t), t \geq 0\}$ with intensity $\lambda(t)$, we define the mean-value function $\Lambda(t)$ by

$$\Lambda(t) = \int_0^t \lambda(s)\, ds.$$

Definition 6 (Poisson Distribution) Poi(λ): *The probability of observing k events is given by*

$$P(X = k) = e^{-\lambda} \frac{\lambda^k}{k!}, \quad k = 0, 1, 2, \ldots$$

The expected value and variance are $E(X) = \mu = \lambda$ and $V(X) = \sigma^2 = \lambda$, respectively.

The Poisson process can be divided into homogeneous with λ being a constant and non-homogeneous with λ being a function of time.

3.3 Task 3 a)

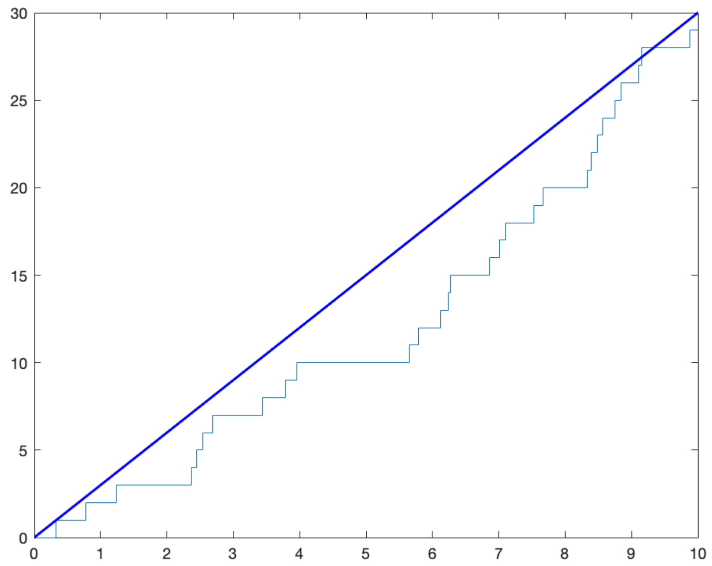
To break down the process into homogeneous and non-homogeneous parts we apply Theorem 4. This theorem states that if we have two independent Poisson processes, say $N_A(t)$ and $N_B(t)$ with rates λ_A and λ_B, then the sum $N_A(t) + N_B(t)$ is equal to $N(t)$ and their rates add up to λ.

In our situation we consider $N(t)$ as being made up of two independent parts: $N_H(t)$ the homogeneous component with a constant rate $\lambda_H = 3$, and $N_N(t)$, the non-homogeneous component with a time-dependent rate $\lambda_N(t) = \frac{\sqrt{4}}{t+2}$. Therefore, we have $\lambda_H + \lambda_N(t) = \lambda(t)$, leading to $\lambda(t) = 3 + \frac{4}{\sqrt{t+2}}$.

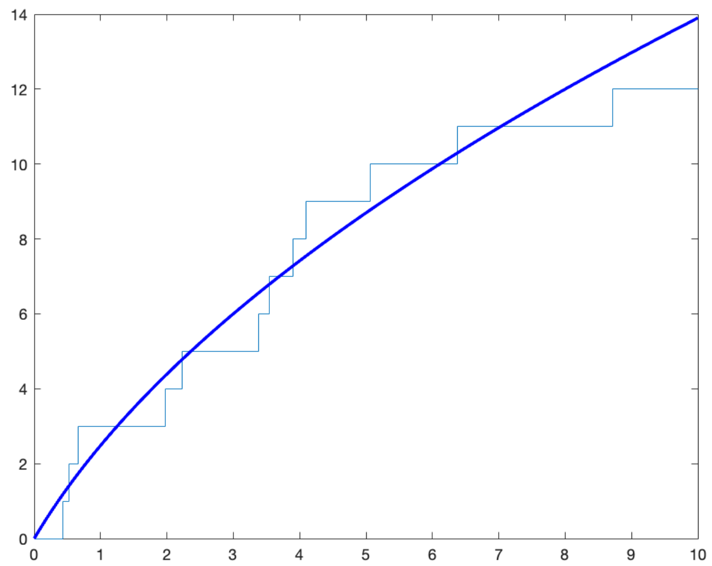
For the homogeneous component we generate a uniformly distributed random number and convert it into an exponentially distributed variable to simulate inter-arrival times.

The non-homogeneous part requires finding the maximum rate $\lambda_{N_{max}}$, since we cannot use inter-arrival times in the same way. This is done by evaluating the rate function's derivative and endpoints to determine the maximum value. Firsty we calculate $\lambda_N(0) = \frac{4}{\sqrt{2}}$ and $\lambda_N(10) = \frac{4}{\sqrt{12}}$. Calculating t^* is unecessery as we see that λ_N decreases as t increases. Considering $t \in [0, 10]$ we get $\lambda_{N_{max}} = \frac{4}{\sqrt{2}}$ where $t = 0$. The simulation for this part is then based on an acceptance-rejection method, using $\lambda_{N_{max}}$ similarly to the constant rate in the homogeneous algorithm.

Results



Figur 2: A simulation of the homogeneous part of the poisson process



Figur 3: A simulation of the non-homogeneous part of the poisson process

Algorithms Task 3 a)

Homogeneous Poisson Process (HPP) Simulation Algorithm

1. Initialize $\lambda = 3$, $T = 10$ $t = 0$, and an empty set S.
2. While $t \leq T$:
   (a) Generate a random number $u \sim \text{Uniform}(0, 1)$.
   (b) Compute the interarrival time $X = -\frac{1}{\lambda}\log(u)$.
   (c) Update $t = t + X$.
   (d) If $t > T$ exit the loop.
   (e) Append t to S.
3. Plot the Poisson process using a step function and overlay a linear function for comparison.

Non-Homogeneous Poisson Process (NHPP) Simulation Algorithm

1. Initialize $\lambda_{\max} = \frac{4}{\sqrt{2}}$, $T = 10$, $t = 0$ and an empty set S.
2. While $t \leq T$:
   (a) Generate a random number $u \sim \text{Uniform}(0, 1)$.
   (b) Compute the interarrival time $X = -\frac{1}{\lambda_{\max}}\log(u)$.
   (c) Update $t = t + X$.
   (d) If $t > T$ exit the loop.
   (e) Generate another random number $u \sim \text{Uniform}(0, 1)$.
   (f) If $u < \frac{\lambda(t)}{\lambda_{\max}}$, append t to S.
3. Plot the NHPP using a step function and overlay a cumulative intensity function for comparison.

3.4 Task 3 b)

To accomplish this task, we utilized the algorithms from task a) to conduct 50 simulations of both the homogeneous process $NH(t)$ and the non-homogeneous process $NN(t)$. We then combined and sequenced the results from $NH(t)$ and $NN(t)$ and plotted the trajectory for each iteration. Additionally the mean-value function was graphed to demonstrate the variation and clustering of the paths. Some initial computations were necessary for this step.

Given that $NH(t)$ is a Poisson distribution with parameter $\lambda_H t$ and considering $\lambda_H = 3$ we have $NH(t) \sim \text{Poisson}(3t)$. The mean value for $NN(t)$ and subsequently for $N(t)$ was obtained using the principles of a Non-homogeneous Poisson process (Def. 5).

$$NN(t) \sim \text{po}(\Lambda(t))$$

Where:

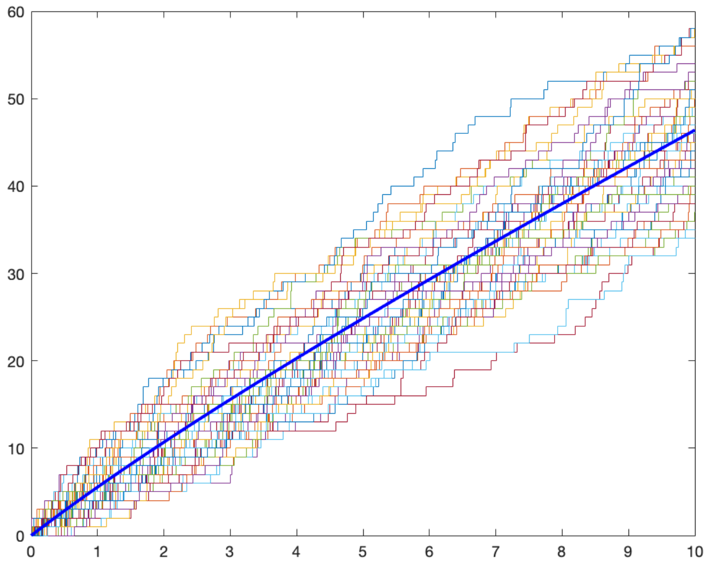
$$\Lambda(t) = \int_0^t \lambda(s)\, ds = 8\sqrt{t+2} - 8\sqrt{2}$$

Moreover, by the superposition property we therefore have:

$$N(t) \sim \text{po}(\Lambda(t) + \lambda t) = \text{po}((8\sqrt{t+2} - 8\sqrt{2}) + 3t)$$

We now have everything needed to simulate the homogeneous and non-homogeneous parts of the process and to plot the mean-value function.

Results



Figur 4: 50 realisations of the poisson process with corresponding mean value function

Algorithm task 3 b)

1. For each simulation j from 1 to n:
   (a) Simulate NHPP:
      i. Initialize an empty set $NHPP$ and $t = 0$.
      ii. While $t \leq T$:
         A. Generate a random number $u \sim \text{Uniform}(0, 1)$.
         B. Compute the interarrival time $X = -\frac{1}{\lambda_{\max}} \log(u)$.
         C. Update $t = t + X$.
         D. If $t > T$ exit the loop.
         E. Generate another random number $u \sim \text{Uniform}(0, 1)$.
         F. If $u < \frac{\lambda_{\text{NHPP}}(t)}{\lambda_{\max}}$, append t to $NHPP$.
   (b) Simulate HPP:
      i. Initialize an empty set HPP and $t = 0$.
      ii. While $t \leq T$:
         A. Generate a random number $u \sim \text{Uniform}(0, 1)$.
         B. Compute the interarrival time $X = -\frac{1}{\lambda_{\text{HPP}}} \log(u)$.
         C. Update $t = t + X$.
         D. If $t > T$ exit the loop.
         E. Append t to HPP.
   (c) Combine and sort the events from both processes into a single set S.
   (d) Plot the combined process using a step function.
2. Overlay a cumulative intensity function for comparison.

3.5 Task 3 c)

In this task we need to combine the homogeneous process with the non- homogeneous process by using Theorem 4 (superposition property): NN(t) + NH(t) = N(t) and $\lambda_N + \lambda_H = \lambda(t)$.

For our Poisson process with intensity function $\lambda(t) = 3 + \frac{4}{\sqrt{t+2}}$ the expected value at time t, $E(N(t))$ is given by the integral of $\lambda(t)$ from 0 to t. Therefore the expected value $E(N(10))$ is:

$$E(N(10)) = \int_0^{10} \lambda(t)\, dt = \int_0^{10} \left(3 + \frac{4}{\sqrt{t+2}}\right) dt$$

Since for a Poisson process the mean equals the variance (Def. 6) the variance $V(N(10))$ is:

$$V(N(10)) = E(N(10))$$

Calculating the integral we find the expected value $E(N(10))$.

The exact value of the expected value $E(N(10))$ is given by the integral:

$$E(N(10)) = -8\sqrt{2} + 16\sqrt{3} + 30$$

The numerical approximation of this value is approximately 46.3991. Thus the theoretical mean and variance $V(N(10))$ are:

$$E(N(10)) \approx 46.3991$$
$$V(N(10)) \approx 46.3991$$

Results

The results of the simulation are as follows:

- Mean: 46.492200
- Variance: 46.547794

- The simulated mean of 46.4922 and variance of 46.5478 are close to the theoretical values of approximately 46.3991 for both mean and variance.This close match indicates a strong agreement between the simulation results and the theoretical model.
- Minor differences are typical due to the randomness in simulations and are in line with the expectations of the law of large numbers.
- In summary the simulation estimates for $E(N(10))$ and $V(N(10))$ closely coincide with the theoretical values validating the accuracy of both the simulation approach and the theoretical model for this Poisson process.

Algorithm task 3 c)

1. Initialize an array 'counts' of size n to store the event counts for each simulation.
2. For each simulation i from 1 to n:
   (a) Simulate HPP:
      i. Initialize $t = 0$.
      ii. While $t \leq T$:
         A. Generate a random number $u \sim \text{Uniform}(0, 1)$.
         B. Compute the interarrival time $X = -\frac{1}{\lambda_{\text{HPP}}} \log(u)$.
         C. Update $t = t + X$.
         D. If $t > T$ exit the loop.
         E. Increment the count for this simulation in 'counts'.
   (b) Simulate NHPP:
      i. Reinitialize $t = 0$.
      ii. While $t \leq T$:
         A. Generate a random number $u \sim \text{Uniform}(0, 1)$.
         B. Compute the interarrival time $X = -\frac{1}{l_{\max}} \log(u)$.
         C. Update $t = t + X$.
         D. If $t > T$ exit the loop.
         E. Generate another random number $u \sim \text{Uniform}(0, 1)$.
         F. If $u < \frac{\lambda_{\text{NHPP}}(t)}{l_{\max}}$ increment the count for this simulation in 'counts'.
3. Calculate the mean and variance of the counts across all simulations.